



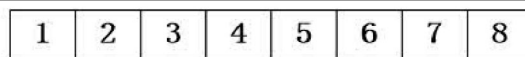
## 7 数据结构与算法基础

### 7.1 线性结构

#### 7.1.1 线性表

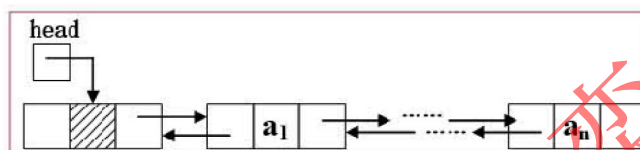
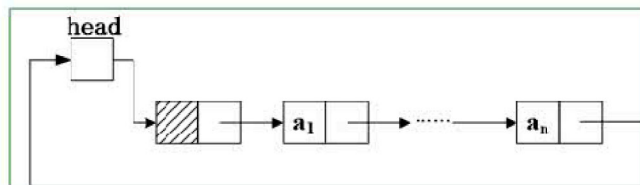
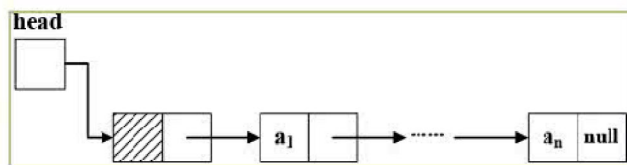
线性表是线性结构（每个元素最多只有一个出度和一个入度，表现为一条线状）的代表，线性表按存储方式分为顺序表和链表，存储形式如下图：

## 顺序表



## 链表

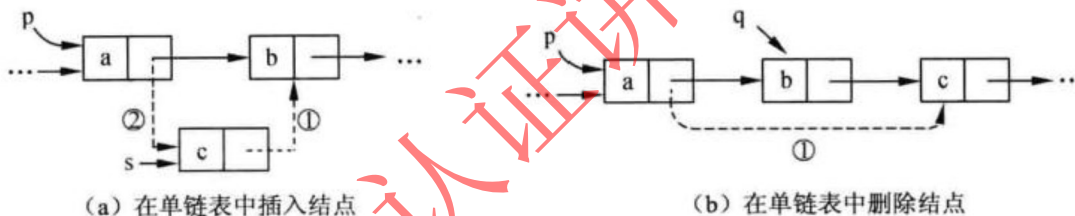
单链表  
循环链表  
双向链表



上图中，顺序表是需要一段连续的内存空间来存放顺序表中的所有元素，这些元素在物理地址上是相邻的。

而链表中的所有元素只是逻辑上相邻，在实际物理存储时处于不同的空闲块中，元素之间通过指针域连接，如上图所示，又可分为单链表、循环链表、双向链表，具体逻辑上的关系都如上图所示。

### 单链表的删除和插入



在上图中  $p$  所指向的节点后插入  $s$  所指向的节点，操作为：

$s \rightarrow \text{next} = p \rightarrow \text{next};$

$p \rightarrow \text{next} = s.$

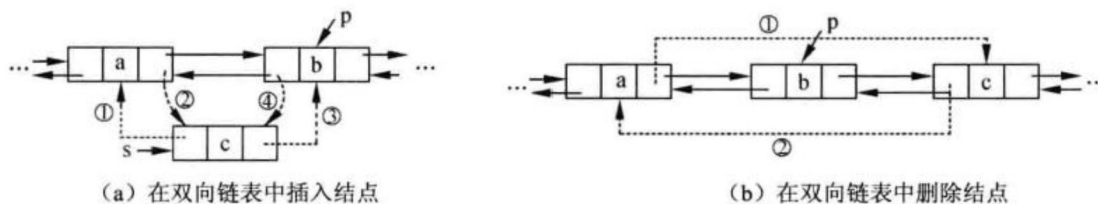
同理，在单链表中删除  $p$  所指向节点的后继节点  $q$  时，操作为：

$q = p \rightarrow \text{next};$

$p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next};$

$\text{free}(p).$

### 双向链表的插入和删除



如上图所示，插入步骤为：

- (1)  $s \rightarrow \text{front} = p \rightarrow \text{front};$
- (2)  $p \rightarrow \text{front} \rightarrow \text{next} = s;$ ，或者表示为  $s \rightarrow \text{front} \rightarrow \text{next} = s;$
- (3)  $s \rightarrow \text{next} = p;$
- (4)  $p \rightarrow \text{front} = s;$

删除节点步骤为：

- (1)  $p \rightarrow \text{front} \rightarrow \text{next} = p \rightarrow \text{next};$
- (2)  $p \rightarrow \text{next} \rightarrow \text{front} = p \rightarrow \text{front}; \text{free}(p);$

顺序存储和链式存储的对比如下图所示：

| 性能类别 | 具体项目 | 顺序存储                    | 链式存储                        |
|------|------|-------------------------|-----------------------------|
| 空间性能 | 存储密度 | =1，更优                   | <1                          |
|      | 容量分配 | 事先确定                    | 动态改变，更优                     |
| 时间性能 | 查找运算 | $O(n/2)$                | $O(n/2)$                    |
|      | 读运算  | $O(1)$ ，更优              | $O([n+1]/2)$ ，最好情况为1，最坏情况为n |
|      | 插入运算 | $O(n/2)$ ，最好情况为0，最坏情况为n | $O(1)$ ，更优                  |
|      | 删除运算 | $O([n-1]/2)$            | $O(1)$ ，更优                  |

在空间方面，因为链表还需要存储指针，因此有空间浪费存在。

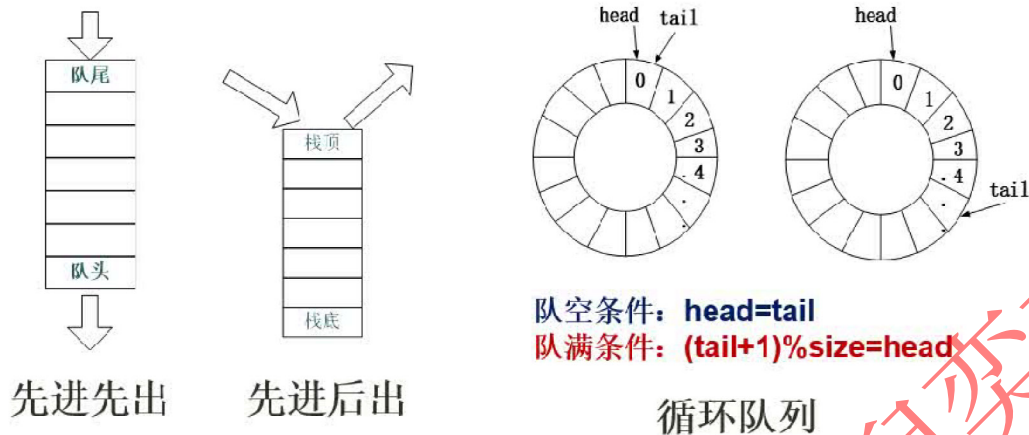
在时间方面，由顺序表和链表的存储方式可知，当需要对元素进行破坏性操作（插入、删除）时，链表效率更高，因为其只需要修改指针指向即可，而顺序表因为地址是连续的，当删除或插入一个元素后，后面的其他节点位置都需要变动。

而当需要对元素进行不改变结构操作时（读取、查找），顺序表效率更高，因为其物理地址是连续的，如同数组一般，只需按索引号就可快速定位，而链表需要从头节点开始，一个个的查找下去。

## 7.1.2 栈和队列

队列、栈结构如下图，队列是先进先出，分队头和队尾；

栈是先进后出，只有栈顶能进出。



循环队列中，头指针指向第一个元素，尾指针指向最后一个元素的下一个位置，因此，当队列空时， $head = tail$ ，当队列满时， $head = tail$ ，这样就无法区分了，因此，一般将队列少存一个元素，这样，队列满时的条件就变为  $tail + 1 = head$ ，而考虑是循环队列，必须要除以最大元素数来取余数，即  $(tail + 1) \% size = head$ ，如上图右边所示两个公式。循环队列的长度公式为  $(Q.tail - Q.head) \% size$ 。

**优先队列：**元素被赋予优先级。当访问元素时，具有最高优先级的元素最先删除。使用堆来存储，因为其不是按照元素进队列的顺序来决定的。

## 7.1.3 串

字符串是一种特殊的线性表，其数据元素都为字符。

**空串：**长度为 0 的字符串，没有任何字符。

**空格串：**由一个或多个空格组成的串，空格是空白字符，占一个字符长度。

**子串：**串中任意长度的连续字符构成的序列称为子串。含有子串的串称为主串，空串是任意串的子串。

**串的模式匹配算法：**子串的定位操作，用于查找子串在主串中第一次出现的位置的算法。

### 模式匹配算法

**基本的模式匹配算法：**也称为布鲁特一福斯算法，其基本思想是从主串的第 1 个字符起与模式串的第 1 个字符比较，若相等，则继续逐个字符进行后续的比较；否则从主串中的第 2 个字符起与模式串的第 1 个字符重新比较，直至模式串中每个字符依次和主串中的一个连续的字符序列相等时为止，此时称为匹配成功，否则称为匹配失败。



**KMP 算法：**对基本模式匹配算法的改进，其改进之处在于：每当匹配过程中出现相比较的字符不相等时，不需要回溯主串的字符位置指针，而是利用已经得到的“部分匹配”结果将模式串向右“滑动”尽可能远的距离，再继续进行比较。

## 7.2 数组、矩阵和广义表

### 7.2.1 数组

主要掌握数组存储地址的计算，特别是二维数组，要注意理解，假设每个数组元素占用存储长度为  $len$ ，起始地址为  $a$ ，存储地址计算如下：

| 数组类型           | 存储地址计算                                                                             |
|----------------|------------------------------------------------------------------------------------|
| 一维数组 $a[n]$    | $a[i]$ 的存储地址为： $a+i*len$                                                           |
| 二维数组 $a[m][n]$ | $a[i][j]$ 的存储地址（按行存储）为： $a+(i*n+j)*len$<br>$a[i][j]$ 的存储地址（按列存储）为： $a+(j*m+i)*len$ |

由二维数组公式可知，若  $i=j$ ，则无论按行存储还是按列存储，存储在  $a[i,j]$  之前的元素个数相同，实际答题时可构造一个简单的二维数组，取特殊值验证。

另外，注意，若题目没有特别说明（注意审题），数组元素一般都是从 0 开始计数的，因此上图中公式都是无需 -1 的。要注意区分二维数组行列计算。

### 7.2.2 稀疏矩阵

稀疏矩阵就是一个矩阵中大部分数据都是 0 的矩阵，此时，只需要存储少部分不为 0 的有效数据即可节省大量空间，因此有下述两种稀疏矩阵：

矩阵      特点      存储数组  $B[k]$  与矩阵元素  $a_{ij}$  对应公式；或示意图

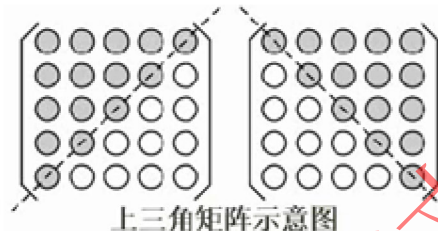
对称矩阵      矩阵  $A_{n*n}$  中的元素特点为  
 $a_{ij}=a_{ji}(1 \leq i, j \leq n)$

$$k = \begin{cases} \frac{i(i-1)}{2} + j & \text{当 } i \geq j \\ \frac{j(j-1)}{2} + i & \text{当 } i < j \end{cases}$$

对角矩阵      矩阵中的非零元素都集中在以主      三对角矩阵示意图：  
对角线为中心的带状区域

$$A_{n \times n} = \begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & & 0 \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \vdots & \vdots & \vdots & \\ & & & a_{i,i-1} & a_{i,i} & a_{i,i+1} \\ & & & & \vdots & \vdots \\ & & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

三角矩阵 矩阵中主对角线上方都为非零元素，下方都为零元素（或相反）。



它们和一维数组之间的对应关系如上公式所示，但在实际考试中，因为都是选择题，可以使用代入法（特殊值法）进行排除，具体见下例和其解析：

### 7.2.3 广义表

广义表是线性表的推广，是由 0 个或多个单元素或子表组成的有限序列。

广义表与线性表的区别：线性表的元素都是结构上不可分的单元素，而广义表的元素既可以单元素，也可以是有结构的表。

广义表一般记为： $LS = (\alpha_1, \alpha_2, \dots, \alpha_n)$

其中 LS 是表名， $\alpha_i$  是表元素，它可以是表（称为子表），也可以是数据元素（称为原子）。其中 n 是广义表的长度（也就是最外层包含的元素个数），n=0 的广义表为空表；而递归定义的重数就是广义表的深度，即定义中所含括号的重数（单边括号的个数，原子的深度为 0，空表的深度为 1）。

head()和 tail(): 取表头（广义表第一个表元素，可以是子表也可以是单元素）和取表尾（广义表中，除了第一个表元素之外的其他所有表元素构成的表，非空广义表的表尾必定是一个表，即使表尾是单元素）操作。

## 7.3 树与二叉树

### 7.3.1 树的相关概念

树结构是一种非线性结构，树中的每一个数据元素可以有两个或两个以上的直接后继元素，用来描述层次结构关系。

树是  $n$  个节点的有限集合 ( $n \geq 0$ )，当  $n=0$  时称为空树，在任一颗非空树中，有且仅有一个根节点。树的基本概念如下：

- (1) 双亲、孩子和兄弟。结点的子树的根称为该结点的孩子；相应地，该结点称为其子结点的双亲。具有相同双亲的结点互为兄弟。
- (2) 结点的度。一个结点的子树的个数记为该结点的度。
- (3) 叶子结点。叶子结点也称为终端结点，指度为 0 的结点。
- (4) 内部结点。度不为 0 的结点，也称为分支结点或非终端结点。除根结点以外，分支结点也称为内部结点。
- (5) 结点的层次。根为第一层，根的孩子为第二层，依此类推，若某结点在第  $i$  层，则其孩子结点在第  $i+1$  层。
- (6) 树的高度。一棵树的最大层数记为树的高度（或深度）。
- (7) 有序（无序）树。若将树中结点的各子树看成是从左到右具有次序的，即不能交换，则称该树为有序树，否则称为无序树。

### 7.3.2 二叉树的特性

二叉树是  $n$  个节点的有限集合，它或者是空树，或者是由一个根节点及两颗互不相交的且分别称为左、右子树的二叉树所组成。与树的区别在于每个根节点最多只有两个孩子结点。

二叉树有一些公式如下，要求掌握，在实际考试中可以用特殊值法验证。

- (1) 二叉树第  $i$  层 ( $i \geq 1$ ) 上最多有  $2^{i-1}$  个结点。  
此性质只要对层数  $i$  进行数学归纳证明即可。
- (2) 高度为  $k$  的二叉树最多有  $2^k - 1$  个结点 ( $k \geq 1$ )。  
由性质 1，每一层的结点数都取最大值  $\sum_{i=1}^k 2^{i-1} = 2^k - 1$  即可。

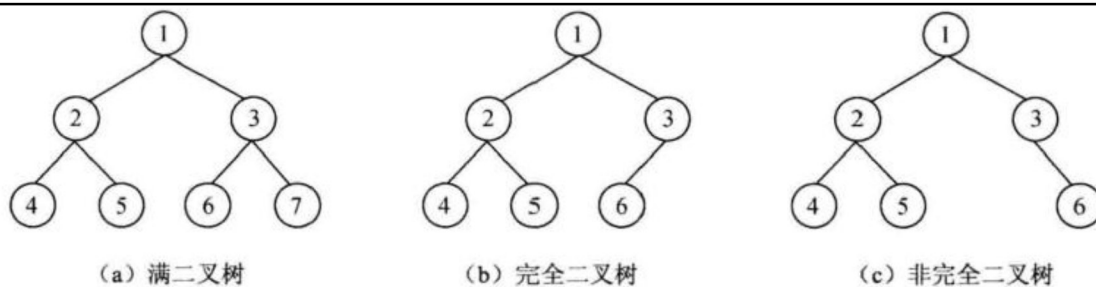
- (3) 对于任何一棵二叉树，若其终端结点数为  $n_0$ ，度为 2 的结点数为  $n_2$ ，则  $n_0 = n_2 + 1$ 。
- (4) 具有  $n$  个结点的完全二叉树的深度为  $\lfloor \log_2 n \rfloor + 1$ 。

(5) 对一棵有  $n$  个结点的完全二叉树的所有结点按层次自上而下、自左至右进行编号，则对于任一结点  $i$  ( $1 \leq i \leq n$ ) 有：

- ① 若  $i=1$ ，则结点  $i$  是二叉树的根，无双亲；若  $i>1$ ，则其双亲为  $\lfloor \frac{i}{2} \rfloor$ 。
- ② 若  $2i > n$ ，则结点  $i$  没有左孩子，否则其左孩子为  $2i$ 。
- ③ 若  $2i+1 > n$ ，则结点  $i$  没有右孩子，否则其右孩子为  $2i+1$ 。

对任何一棵二叉树，若其终端节点数为  $n_0$ ，度为 2 的节点数为  $n_2$ ，则  $n_0 = n_2 + 1$ ，此公式可以画一个简单的二叉树使用特殊值法快速验证，也可以证明如下：设一棵二叉树上叶结点数为  $n_0$ ，单分支结点数为  $n_1$ ，双分支结点数为  $n_2$ ，则总结点数  $= n_0 + n_1 + n_2$ 。在一棵二叉树中，所有结点的分支数(即度数)应等于单分支结点数加上双分支结点数的 2 倍，即总的分支数  $= n_1 + 2n_2$ 。由于二叉树中除根结点以外，每个结点都有唯一的一个分支指向它，因此二叉树中：总的分支数  $=$  总结点数  $- 1$ 。

两种特殊的二叉树如下图所示：



可知，满二叉树每层都是满的。

完全二叉树的  $k-1$  层是满的，第  $k$  层节点从左到右是满的。

### 7.3.3 二叉树的存储结构

#### 二叉树的顺序存储结构

顺序存储，就是用一组连续的存储单元存储二叉树中的节点，按照从上到下，从左到右的顺序依次存储每个节点。

对于深度为  $k$  的完全二叉树，除第  $k$  层外，其余每层中节点数都是上一层的两倍，由此，从一个节点的编号可推知其双亲、左孩子、右孩子节点的编号。假设有编号为  $i$  的节点，则有：

- (1) 若  $i=1$ ，该结点为根结点，无双亲。
- (2) 若  $i>1$ ，该结点的双亲为  $(i+1)/2$ （取整数）。
- (3) 若  $2i \leq n$ ，则该结点的左孩子编号为  $2i$ ，否则无左孩子。
- (4) 若  $2i+1 \leq n$ ，则该结点的右孩子编号为  $2i+1$ ，否则无右孩子。
- (5) 若  $i$  为奇数且不为 1，则该结点左兄弟的编号为  $i-1$ ，否则无左兄弟。
- (6) 若  $i$  为偶数且小于  $n$ ，则该结点右兄弟的编号为  $i-1$ ，否则无右兄弟。

#### 二叉树的链式存储结构

一般用二叉链表来存储二叉树节点，二叉链表中除了该节点本身的数据外，还存储有左孩子结点的指针、右孩子结点的指针，即一个数据+两个指针。

每个二叉链表节点存储一个二叉树节点，头指针则指向根节点。

### 7.3.4 二叉树的遍历

一颗非空的二叉树由根节点、左子树、右子树三部分组成，遍历这三部分，也就遍历了整颗二叉树。这三部分遍历的基本顺序是先左子树后右子树，但根节点顺序可变，以根节点访问的顺序为准有下列三种遍历方式：

先序（前序）遍历：根左右。



中序遍历：左根右。

后序遍历：左右根。

还有层次遍历方法：

层次遍历：按层次，从上到下，从左到右。

反向构造二叉树：

仅仅有前序和后序是无法构造二叉树的，必须要是和中序遍历的集合才能反向构造出二叉树。

构造时，前序和后序遍历可以确定根节点，中序遍历用来确定根节点的左子树节点和右子树节点，而后按此方法进行递归，直至得出结果。

### 7.3.5 线索二叉树

引入线索二叉树是为了保存二叉树遍历时某节点的前驱节点和后继节点的信息，二叉树的链式存储只能获取到某节点的左孩子和右孩子结点，无法获取其遍历时的前驱和后继节点，因此可以在链式存储中再增加两个指针域，使其分别指向前驱和后继节点，但这样太浪费存储空间，考虑下述实现方法：

若  $n$  个节点的二叉树使用二叉链表存储，则必然有  $n+1$  个空指针域，利用这些空指针域来存放节点的前驱和后继节点信息，为此，需要增加两个标志，以区分指针域存放的到底是孩子结点还是遍历节点，如下：

| ltag | lchild | data | rchild | rtag |
|------|--------|------|--------|------|
|------|--------|------|--------|------|

$ltag = \begin{cases} 0 & \text{lchild 域指示结点的左孩子} \\ 1 & \text{lchild 域指示结点的直接前驱} \end{cases}$

$rtag = \begin{cases} 0 & \text{rchild 域指示结点的右孩子} \\ \bullet & \\ 1 & \text{rchild 域指示结点的直接后继} \end{cases}$

若二叉树的二叉链表采用上述结构，则称为线索链表，其中指向前驱、后继节点的指针称为线索，加上线索的二叉树称为线索二叉树。



### 7.3.6 最优二叉树（哈夫曼树）

最优二叉树又称为哈夫曼树，是一类带权路径长度最短的树，相关概念如下：

**树的路径长度：**根节点到达叶子节点的层次距离（或者说经过的分支数）。

**权：**叶子节点代表的值。

**带权路径长度：**叶子节点的路径长度乘以该叶节点的值。

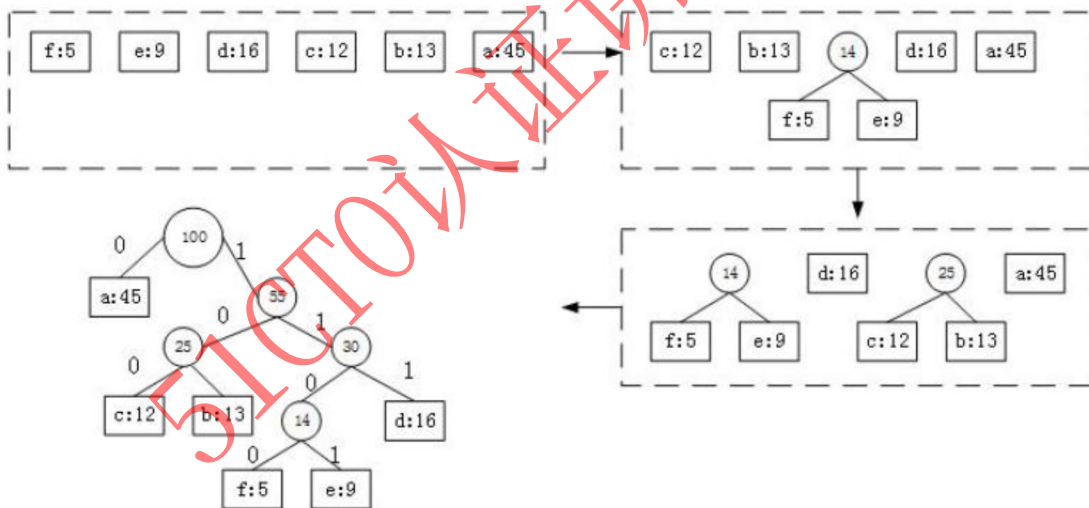
**树的带权路径长度（树的代价）：**树的所有叶子节点的带权路径长度之和。

**哈夫曼树的求法：**给出一组权值，将其中两个最小的权值作为叶子节点，其和作为父节点，组成二叉树，而后删除这两个叶子节点权值，并将父节点的值添加到该组权值中。重复进行上述步骤，直至所有权值都被使用完。

构造出的哈夫曼树中，所有初始给出的权值都作为叶子节点，此时，求出每个叶子节点的带权路径长度，而后相加，就是**树的带权路径长度，这个长度是最小的。**

**若需要构造哈夫曼编码**（要**保证左节点值小于右节点的值**，才是标准的哈夫曼树），将标准哈夫曼树的左分支设为**0**，右分支设为**1**，写出每个叶节点的编码，会发现，哈夫曼编码前缀不同，因此不会混淆，同时也是最优编码。

构造过程如下：



### 7.3.7 树和森林

#### 树的存储结构

**双亲表示法：**用一组连续的地址单元存储树的节点，并在每个节点中附带一个指示器，指出其双亲节点所在数组元素的下标。

孩子表示法：在存储结构中用指针指示出节点的每个孩子，为树中每个节点的孩子建立一个链表。

孩子兄弟表示法：又称为二叉链表表示法，为每个存储节点设置两个指针域，分别指向该节点的第一个孩子和下一个兄弟节点。

### 树和森林的遍历

由于树中每个节点可能有多个子树，因此遍历树的方法有两种：

先根遍历：先访问根节点，再依次遍历根的各颗子树。

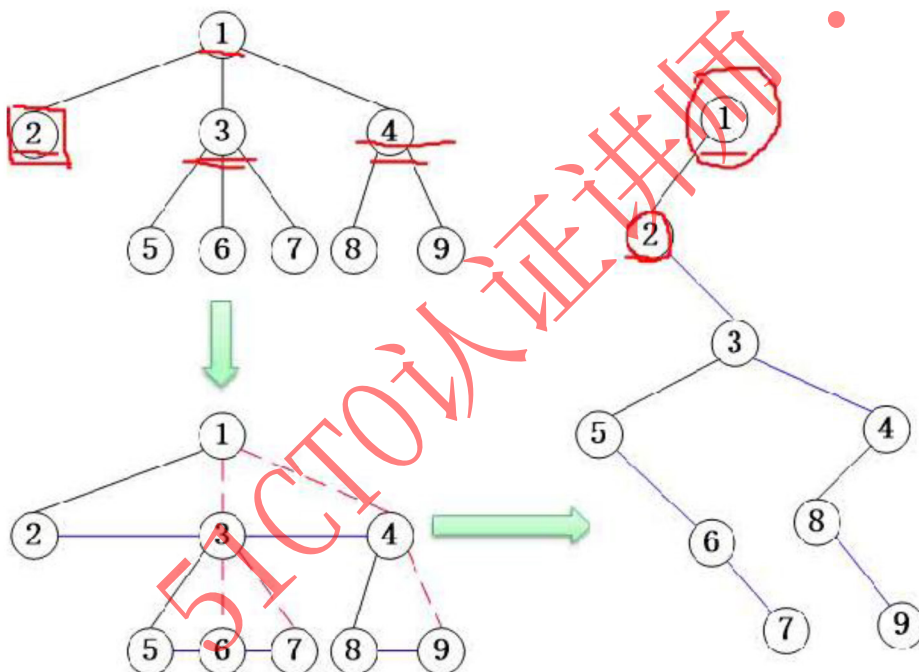
后根遍历：先遍历根的各颗子树，再访问根节点。

森林中有很多棵树，森林的遍历方法也分为两种，与树的遍历类似，就是对森林中的每棵树都依次做先根遍历或后根遍历。

### 树和二叉树的转换

规则是：树的最左边节点作为二叉树的左子树，树的其他兄弟节点作为二叉树的右子树节点。

示例如下图：采用连线法，将最左边节点和其兄弟节点都连接起来，而原来的父节点和兄弟节点的连线则断开，这种方法最简单，要求掌握。



### 7.3.8 查找（排序）二叉树

查找二叉树上的每个节点都存储一个值，且每个节点的**所有左孩子结点值都小于父节点值**，而**所有右孩子结点值都大于父节点值**，是一个有规律排列的二叉树，这种数据结构可以方便查找、插入等数据操作。

二叉排序树的查找效率取决于二叉排序树的深度，对于结点数相同的二叉排序树，平衡二叉树的深度最小，而单枝树的深度是最大的，故效率最差。

### 7.3.9 平衡二叉树

前面讲过查找(排序)二叉树，特点是所有左子树值小于根节点值，所有右子树值大于根节点值，而这个特点可以构造出多个不同的二叉树，并不唯一，因此提出平衡二叉树的概念，在查找二叉树特点的基础上，要求每个节点的平衡度只能为 0 或 1 或 -1。

节点的左右子树深度就是其左右子树各自的层数，而后将左子树深度减去右子树深度，就得到了该节点的平衡度。因此，平衡二叉树就是任意左右子树层次相差不超过 1。

## 7.4 图

### 7.4.1 图的基本概念

图也是一种非线性结构，图中任意两个节点间都可能存在直接关系。相关定义如下：

**无向图：**图的结点之间连接线是没有箭头的，不分方向。

**有向图：**图的结点之间连接线是箭头，区分 A 到 B，和 B 到 A 是两条线。

**完全图：**无向完全图中，节点两两之间都有连线，n 个结点的连线数为  $(n-1)+(n-2)+\cdots+1 = n*(n-1)/2$ ；  
有向完全图中，节点两两之间都有互通的两个箭头，n 个节点的连线数为  $n*(n-1)$ 。

**度、出度和入度：**顶点的度是关联与该顶点的边的数目。在有向图中，顶点的度为出度和入度之和。出度是以该顶点为起点的有向边的数目。入度是以该顶点为终点的有向边的数目。

**路径：**存在一条通路，可以从一个顶点到达另一个顶点，有向图的路径也是有方向的。

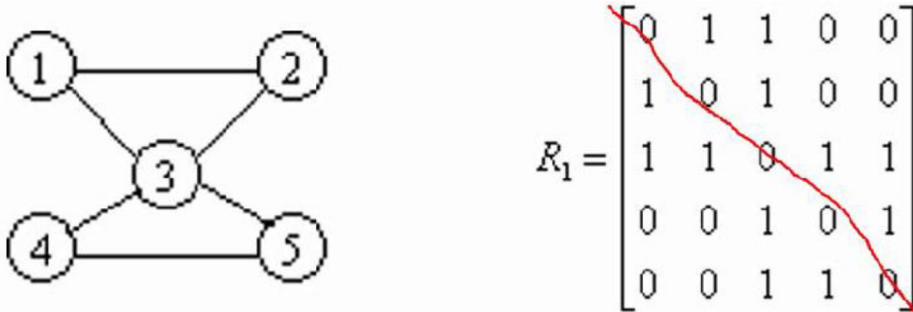
**连通图和连通分量：**针对无向图。若从顶点 v 到顶点 u 之间是有路径的，则说明 v 和 u 之间是连通的，若无向图中任意两个顶点之间都是连通的，则称为连通图。无向图 G 的极大连通子图称为其连通分量。

**强连通图和强连通分量：**针对有向图。若有向图任意两个顶点间都互相存在路径，即存在 v 到 u，也存在 u 到 v 的路径，则称为强连通图。有向图中的极大连通子图称为其强连通分量。

**网：**边带权值的图称为网。

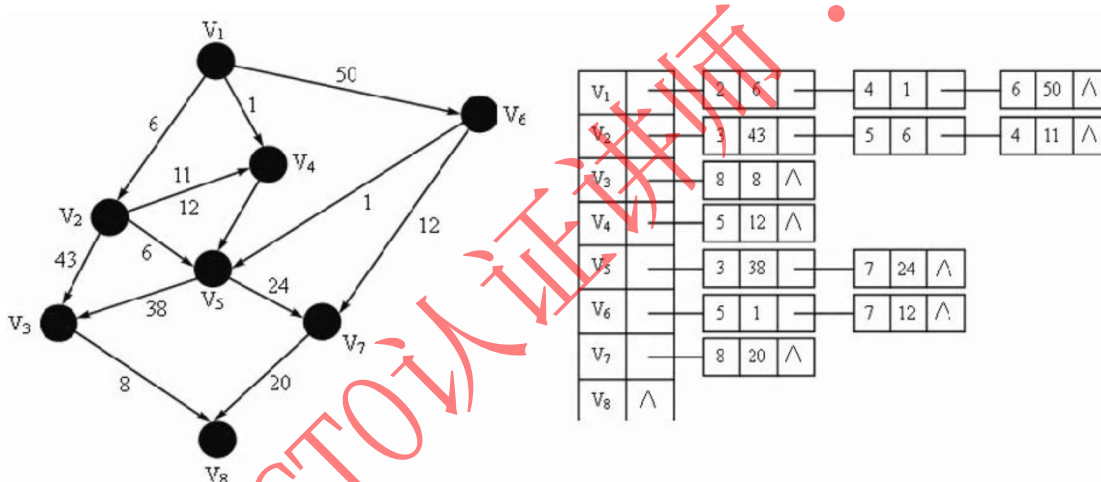
## 7.4.2 图的存储

**邻接矩阵：**假设一个图中有  $n$  个节点，则使用  $n$  阶矩阵来存储这个图中各节点的关系，规则是若节点  $i$  到节点  $j$  有连线，则矩阵  $R_{i,j}=1$ ，否则为 0，示例如下图所示：



由上图可知，如果是一个无向图，肯定是沿对角线对称的，只需要存储上三角或者下三角就可以了，而有向图则不一定对称。

**邻接链表：**用到了两个数据结构，先用一个一维数组将图中所有顶点存储起来，而后，对此一维数组的每个顶点元素，使用链表挂上和其有连线关系的结点的编号和权值，示例如下图所示：



**存储特点：**图中的顶点数决定了邻接矩阵的阶和邻接表中的单链表数目，无论是对有向图还是无向图，边数的多少决定了单链表中的结点数，而不影响邻接矩阵的规模，因此采用何种存储方式与有向图、无向图没有区别，要看图的边数和顶点数，完全图适合采用邻接矩阵存储。

## 7.4.3 图的遍历

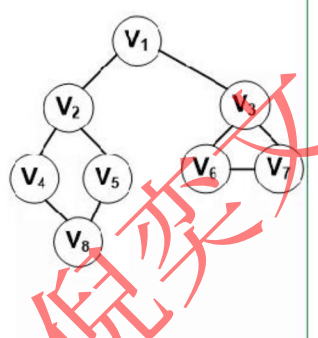
图的遍历是指从图的任意节点出发，沿着某条搜索路径对图中所有节点进行访问且只访问一次，分为以下两种方式：

**深度优先遍历：**从任一顶点出发，遍历到底，直至返回，再选取任一其他节点出发，重复这个过程直至遍历完整个图；



**广度优先遍历：**先访问完一个顶点的所有邻接顶点，而后再依次访问其邻接顶点的所有邻接顶点，类似于层次遍历。

在实际应用中，一般给出邻接表或者邻接矩阵，来求遍历，这时，可以先画出图，再求最保险，当然简单的结构也可以利用存储结构特点来求。

| 遍历方法 | 说明                                                                             | 示例                                                      | 图例                                                                                 |
|------|--------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------|
| 深度优先 | 1.首先访问出发顶点V<br>2.依次从V出发搜索V的任意一个邻接点W；<br>3.若W未访问过，则从该点出发继续深度优先遍历<br>它类似于树的前序遍历。 | $V_1, V_2,$<br>$V_4, V_8,$<br>$V_5, V_3,$<br>$V_6, V_7$ |  |
| 广度优先 | 1.首先访问出发顶点V<br>2.然后访问与顶点V邻接的全部未访问顶点W、X、Y...；<br>3.然后再依次访问W、X、Y...邻接的未访问的顶点；    | $V_1, V_2,$<br>$V_3, V_4,$<br>$V_5, V_6,$<br>$V_7, V_8$ |                                                                                    |

#### 7.4.4 图的最小生成树

假设有  $n$  个节点，那么这个图的**最小生成树有  $n-1$  条边**（不会形成环路，是树非图），这  $n-1$  条边应该会将所有顶点都连接成一个树，并且**这些边的权值之和最小**，因此称为最小生成树。

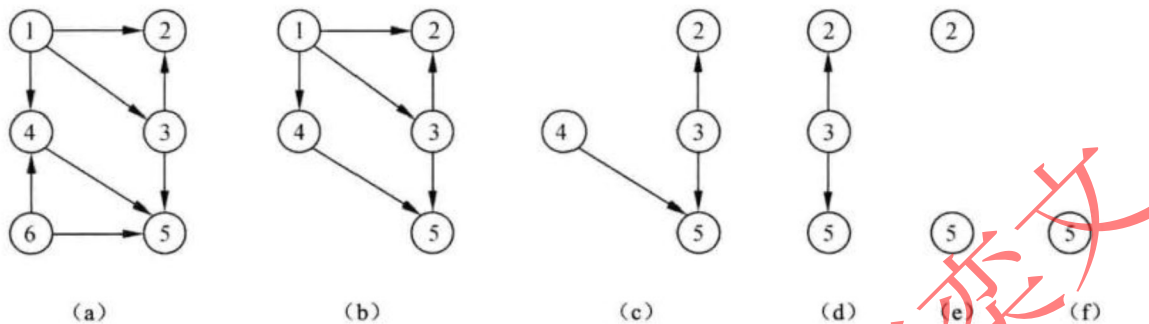
因此，本质就是找出图中权值最小的  $n-1$  条边，使图中所有节点连接成一棵树，共有下列两种算法：

**普里姆算法：**从任意顶点出发，找出与其邻接的边权值最小的，此时此边的另外一个顶点自动加入树集合中，而后再从这个树集合的所有顶点中找出与其邻接的边权值最小的，同样此边的另外一个顶点加入树集合中，依次递归，直至图中所有顶点都加入树集合中，此时此树就是该图的最小生成树。

**克鲁斯卡尔算法**（推荐）：这个算法是**从边出发**的，因为本质是选取权值最小的  $n-1$  条边，因此，就将边按权值大小排序，依次选取权值最小的边，直至囊括所有节点，要注意，每次选边后要检查不能形成环路。

## 7.4.5 拓扑序列

将有向图的有向边作为活动开始的顺序，若图中一个节点入度为 0，则应该最先执行此活动，而后删除掉此节点和其关联的有向边，再去找图中其他没有入度的结点，执行活动，依次进行，示例如下（有点类似于进程的前趋图原理）：



## 7.4.6 关键路径

有一种工程管理的图，边代表活动，边上权值为该活动执行时间，节点代表一个时间节点，图中节点相关概念如下：

**最早开始时间：**所有前驱活动的最早完成时间（实际是所有前驱活动都完成，即取所有前驱活动中耗时最长的路径时间）即是该活动的最早开始时间。

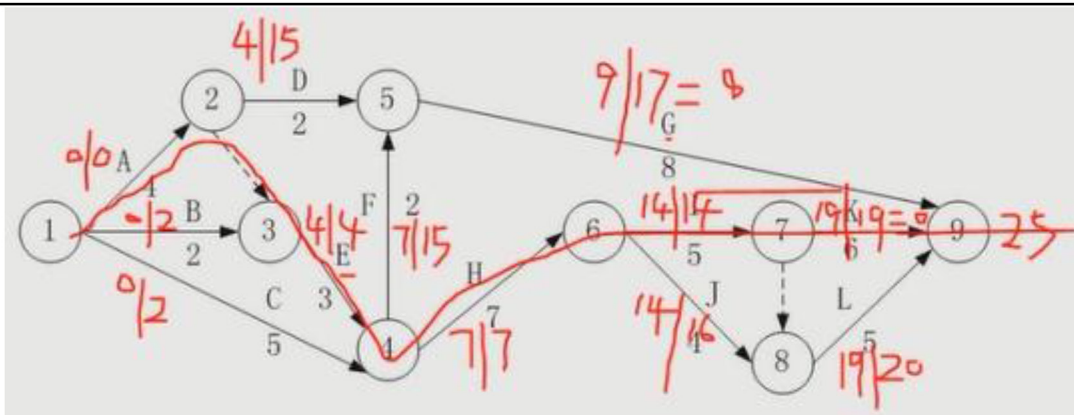
**最早完成时间：**最早开始时间+活动本身时间。

由上面两个，最终可以求出**关键路径（项目中耗时最长的一条线路）**，即项目总工期，由项目总工期可以从终点开始向前求最晚开始时间：

**最晚开始时间：**在不影响总工期的情况下，某活动的最晚开始时间，由项目总工期依次向前减掉中间活动的耗时，若有多个后继活动，取最大的相减。

**松弛时间：**某活动的最晚开始时间-最早开始时间（或最晚完成时间-最早完成时间），也即该活动最多可以晚开始多少天。

具体分析如下（最早开始时间|最晚开始时间，红线为关键路径）：



## 7.5 算法特性

### 7.5.1 算法的特性

算法的五个特点如下所示：要注意的是，可以没有输入，但是必须要有输出，否则算法是没有意义的。

(1) 有穷性。一个算法必须总是（对任何合法的输入值）在执行有穷步之后结束，且每一步都可在有穷时间内完成。

(2) 确定性。算法中的每一条指令必须有确切的含义，理解时不会产生二义性。并且在任何条件下，算法只有唯一的一条执行路径，即对于相同的输入只能得出相同的输出。

(3) 可行性。一个算法是可行的，即算法中描述的操作都可以通过已经实现的基本运算执行有限次来实现。

(4) 输入。一个算法有零个或多个输入，这些输入取自于某个特定的对象的集合。

(5) 输出。一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

### 7.5.2 算法的复杂度

**时间复杂度**是指程序运行从开始到结束所需要的时间。通常分析时间复杂度的方法是从算法中选取一种对于所研究的问题来说是基本运算的操作，以该操作重复执行的次数作为算法的时间度量。一般来说，算法中原操作重复执行的次数是规模 $n$ 的某个函数 $T(n)$ 。由于许多情况下要精确计算 $T(n)$ 是困难的，因此引入了渐进时间复杂度在数量上估计一个算法的执行时间。其定义如下：

如果存在两个常数 $c$ 和 $m$ ，对于所有的 $n$ ，当 $n \geq m$ 时有 $f(n) \leq cg(n)$ ，则有 $f(n) = O(g(n))$ 。也就是说，随着 $n$ 的增大， $f(n)$ 渐进地不大于 $g(n)$ 。例如，一个程序的实际执行时间为 $T(n) = 3n^3 + 2n^2 + n$ ，则 $T(n) = O(n^3)$ 。

常见的对算法执行所需时间的度量：

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n)$